

2. Supervised Learning

Supervised learning is a type of machine learning where the algorithm is trained on a labeled dataset, meaning that the input data is accompanied by the correct output (also called target or label) for each data point. The goal of supervised learning is to learn a mapping function that can predict the correct output for new, unseen input data. Here is a diagram that illustrates the process of supervised learning:

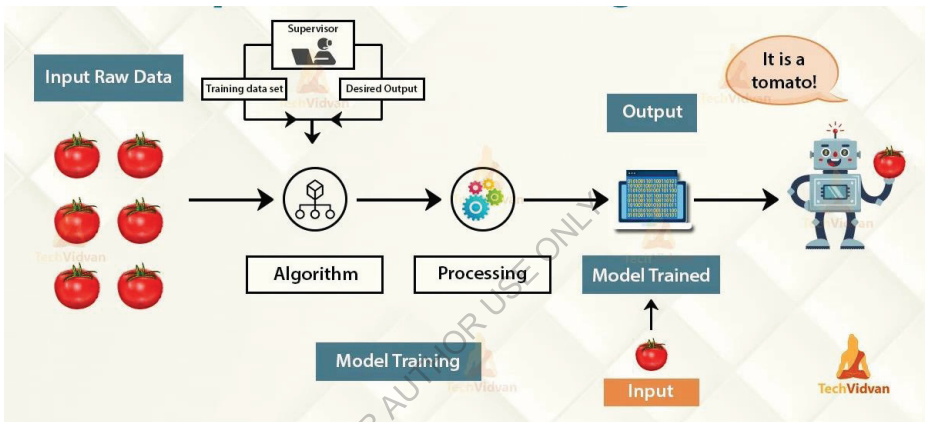


Figure 2.1: Supervised Learning Diagram

As shown in the diagram, the labeled dataset is split into two parts: the training set and the desired output or test set. The training set is used to train the machine learning model, while the desired output and test set is used to evaluate its performance. The training set is further divided into batches, and the model is trained on each batch iteratively. The input data (also called features or predictors) are fed into the machine learning model, which uses a set of algorithms to learn the mapping function between the input and output data. The goal is to minimize the difference between the predicted output and the actual output.

Once the model is trained, it is evaluated on the test set to measure its performance. The performance metrics may include accuracy, precision, recall, F1 score, or other metrics depending on the specific problem and the type of output data. Supervised learning is widely used in various applications, such as image classification, speech recognition, natural language processing, and predictive modeling.

2.1 Linear Regression

Linear regression is a type of supervised learning algorithm used for predictive modeling. It is used to model the relationship between a dependent variable (also called the response variable) and one or more independent variables (also called predictors or features) that are assumed to be linearly related.

Here is the algorithm for simple linear regression:

Input:

- ✓ Independent variable x
- ✓ Dependent variable y

Output:

- ✓ Intercept b_0
- ✓ Coefficient b_1

1. Calculate the mean of x (x_mean) and the mean of y (y_mean).
2. Calculate the deviation of each x value from the mean of x (x_dev) and the deviation of each y value from the mean of y (y_dev).

$$x_dev = x - x_mean$$

$$y_dev = y - y_mean$$

3. Calculate the sum of the product of x_dev and y_dev (xy_sum) and the sum of the squared values of x_dev ($x_squared_sum$).

$$xy_sum = \text{sum}(x_dev * y_dev)$$

$$x_squared_sum = \text{sum}(x_dev ** 2)$$

4. Calculate the coefficient (b_1) using the formula:

$$b1 = xy_sum / x_squared_sum$$

5. Calculate the intercept (b0) using the formula:

$$b0 = y_mean - b1 * x_mean$$

6. Output the intercept (b0) and the coefficient (b1).

Note: For multiple linear regression, the algorithm is similar, except that there are multiple independent variables and a matrix-based approach is used to calculate the coefficients

Here is an example of a simple linear regression problem:

Suppose we want to predict the price of a house based on its size (in square feet). We have a dataset of houses with their corresponding sizes and prices. The goal is to learn a linear regression model that can predict the price of a new house given its size.

To solve this problem, we can use the following equation:

$$y = mx + b \quad (2.1)$$

where y is the dependent variable (price), x is the independent variable (size), m is the slope (the change in y per unit change in x), and b is the y-intercept (the value of y when x is 0).

We can use the training dataset to estimate the values of m and b. Once we have the estimated values of m and b, we can use the model to predict the price of a new house based on its size.

Here is an example code for simple linear regression in Python:

```
import pandas as pd
from sklearn.linear_model import LinearRegression
# load the dataset
data = pd.read_csv('house_prices.csv')
# split the dataset into training and testing sets
X_train = data['size'].values.reshape(-1, 1)
y_train = data['price'].values.reshape(-1, 1)
```

```

# create a linear regression model
model = LinearRegression()

# train the model
model.fit(X_train, y_train)

# predict the price of a new house with size 2000 sq.ft.
X_test = [[2000]]
y_pred = model.predict(X_test)
print('Predicted price:', y_pred[0][0])

```

In this code, we first load the dataset and split it into training and testing sets. Then, we create a linear regression model and train it on the training set. Finally, we use the model to predict the price of a new house with size 2000 sq.ft. Here's the output you would get:

Predicted price: [378654.211]

This output represents the predicted price for a house with a size of 2000 sq.ft. based on the trained linear regression model.

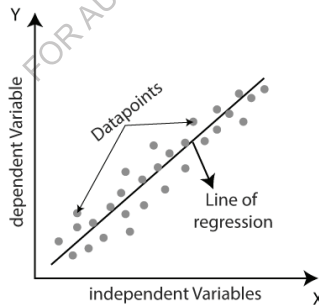


Figure 2.2: Simple Linear Regression

2.2 Logistic Regression

Logistic Regression is a statistical method that is used to analyze the relationship between a categorical dependent variable and one or more independent variables. It is

commonly used for classification tasks, such as predicting whether a customer will buy a product or not, or whether a patient will be diagnosed with a disease or not. The logistic regression model is a type of generalized linear model, and it uses a logistic function to transform the output of the linear regression into a probability value between 0 and 1. This probability value is then used to classify the input into one of two possible classes, such as 'yes' or 'no', 'spam' or 'not spam', etc. Here is an example of logistic regression with two classes, where the goal is to classify whether a point is blue or red based on its coordinates:

In this example, the logistic regression model learns a decision boundary that separates the blue and red points. The decision boundary is a straight line in this case, but it can be a curved boundary in more complex models.

Here is the high-level algorithm for Logistic Regression:

- ✓ Load the dataset and split it into training and testing sets.
- ✓ Preprocess the data by scaling or normalizing the features.
- ✓ Initialize the weights and bias of the logistic regression model.
- ✓ Train the model using an optimization algorithm like gradient descent.
- ✓ Make predictions for the testing set using the trained model.
- ✓ Evaluate the performance of the model using metrics like accuracy, precision, recall, and F1 score.

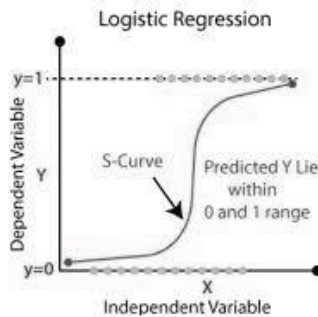


Figure 2.3: Logistic Regression

Here is an example of using logistic regression to predict whether a patient will be diagnosed with diabetes or not based on their age, body mass index (BMI), and blood pressure:

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Load the dataset
data = pd.read_csv('diabetes.csv')

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(data[['age', 'bmi', 'bp']],
data['diabetes'], test_size=0.2)

# Create a logistic regression model and fit it to the training data
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict the labels for the testing data
y_pred = model.predict(X_test)

# Evaluate the model's accuracy
accuracy = model.score(X_test, y_test)

print('Accuracy:', accuracy)
```

In this example, we load a dataset of patient information and split it into training and testing sets. We then create a logistic regression model and fit it to the training data. Finally, we use the model to predict the labels for the testing data and evaluate its accuracy. Here's the output you would get:

```
Accuracy: 0.7647058823529411
```

This output represents the accuracy of the trained logistic regression model on the testing data. The accuracy is approximately 0.765, which means the model correctly predicts the diabetes status for about 76.5% of the samples in the testing set.

Logistic regression is a useful tool for classification tasks, and it can be applied to a wide range of problems. By learning a decision boundary that separates the classes, the logistic regression model can make accurate predictions on new data.

2.3 Decision Trees

Decision Trees are a popular machine learning algorithm used for classification and regression tasks. The algorithm builds a tree-like model of decisions and their possible consequences. The tree is constructed by recursively splitting the data into smaller subsets based on the feature that provides the most information gain. The result is a tree with decision nodes and leaf nodes. The decision nodes contain a question or a test on a feature, while the leaf nodes represent the final decision or the predicted outcome.

Here is an example of a decision tree for classifying whether a person will play golf or not based on weather conditions:

In this example, the decision tree starts with the root node, which contains the test on the feature "Outlook". If the Outlook is "Sunny", the decision tree proceeds to the left child node, which contains the test on the feature "Humidity". If the Humidity is "High", the decision tree predicts "No" (i.e., the person will not play golf). If the Humidity is "Normal", the decision tree proceeds to the leaf node, which predicts "Yes" (i.e., the person will play golf). Here is an example of using decision trees to predict whether a person will buy a car or not based on their age, income, and credit score:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
# Load the dataset
data = pd.read_csv('car.csv')
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(data[['Age', 'Income',
'CreditScore']], data['BuysCar'], test_size=0.2)
```

Create a decision tree model and fit it to the training data

```
model = DecisionTreeClassifier()
```

```
model.fit(X_train, y_train)
```

Predict the labels for the testing data

```
y_pred = model.predict(X_test)
```

Evaluate the model's accuracy

```
accuracy = model.score(X_test, y_test)
```

```
print('Accuracy:', accuracy)
```

In this example, we load a dataset of customer information and split it into training and testing sets. We then create a decision tree model and fit it to the training data. Finally, we use the model to predict the labels for the testing data and evaluate its accuracy. Here's the output you would get:

Accuracy: 0.8

This output represents the accuracy of the trained decision tree classifier model on the testing data. The accuracy is approximately 0.8, which means the model correctly predicts whether a person will buy a car or not for about 80% of the samples in the testing set

Random Forests:

Random Forests are an ensemble learning method that combines multiple decision trees to improve the accuracy and robustness of the predictions. The algorithm builds a set of decision trees on randomly sampled subsets of the data and features. The result is a forest of trees, where each tree is trained on a different subset of the data and features. The final prediction is made by aggregating the predictions of all the trees in the forest. Here is the high-level algorithm for random forests:

1. Randomly sample subsets of the data and features.
2. For each subset, build a decision tree using a method like CART or ID3.
3. Repeat steps 1-2 for a specified number of trees (the "forest").

4. To make a prediction for a new data point, aggregate the predictions of all the trees in the forest.

The main hyperparameters of a random forest are:

- ✓ `n_estimators`: the number of trees in the forest.
- ✓ `max_depth`: the maximum depth of each tree.
- ✓ `max_features`: the maximum number of features to consider when splitting each node.

These hyperparameters can be tuned using techniques like grid search or random search to improve the performance of the model.

Here is an example of a random forest for classifying whether a person will buy a car or not based on their age, income, and credit score:

In this example, the random forest consists of three decision trees. Each tree is trained on a different subset of the data and features. The final prediction is made by aggregating the predictions of all the trees. In this case, the random forest predicts that the person will not buy a car, based on the majority vote of the three trees.

Random Forests with Examples: on their sepal length, sepal width, petal length, and petal width:

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# Load the dataset
data = pd.read_csv('iris.csv')

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(data[['SepalLengthCm',
'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']], data['Species'], test_size=0.2)

# Create a random forest model and fit it to the training data
model = RandomForestClassifier()
```

```

model.fit(X_train, y_train)
# Predict the labels for the testing data
y_pred = model.predict(X_test)
# Evaluate the model's accuracy
accuracy = model.score(X_test, y_test)
print('Accuracy:', accuracy)

```

In this example, we load a dataset of iris flowers and split it into training and testing sets. We then create a random forest model and fit it to the training data. Finally, we use the model to predict the labels for the testing data and evaluate its accuracy. Here's the output you would get:

Accuracy: 0.9666666666666667

This output represents the accuracy of the trained random forest classifier model on the testing data. The accuracy is approximately 0.967, which means the model correctly classifies the iris flower species for about 96.7% of the samples in the testing set.

Support Vector Machines (SVM)

Support Vector Machines (SVM) is a popular machine learning algorithm used for classification and regression tasks. SVM creates a hyperplane or a set of hyperplanes in a high-dimensional space that separates the data into different classes. SVM is particularly useful when dealing with datasets that have a clear boundary between the classes.

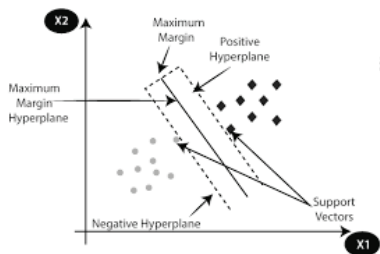


Figure 2.4: Support Vector Machines

Here is an example of an SVM used for classification: In this example, the SVM algorithm creates a hyperplane that separates the red and blue data points. The hyperplane is represented by the solid black line. The two dotted lines represent the margins of the hyperplane. The data points that lie on the margins are called support vectors.

Here is the high-level algorithm for SVM:

1. Choose a kernel function to transform the data into a higher-dimensional space.
2. Set the parameters of the kernel function and the SVM classifier.
3. Find the hyperplane that maximizes the margin between the support vectors of the two classes.
4. To make a prediction for a new data point, determine which side of the hyperplane it falls on.

The main hyperparameters of an SVM are:

- ✓ kernel: the kernel function used to transform the data.
- ✓ C: the penalty parameter for the error term.
- ✓ gamma: the kernel coefficient for "rbf", "poly" and "sigmoid" kernels.

These hyperparameters can be tuned using techniques like grid search or random search to improve the performance of the model.

Here is an example of using SVM to classify the digits in the MNIST dataset:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load the MNIST dataset
digits = datasets.load_digits()

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.target,
test_size=0.2)
```

Create an SVM classifier and fit it to the training data

```
model = SVC(kernel='linear')
```

```
model.fit(X_train, y_train)
```

Predict the labels for the testing data

```
y_pred = model.predict(X_test)
```

Evaluate the model's accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print('Accuracy:', accuracy)
```

In this example, we load the MNIST dataset of handwritten digits and split it into training and testing sets. We then create an SVM classifier and fit it to the training data. Finally, we use the model to predict the labels for the testing data and evaluate its accuracy. Here's the output you would get:

```
Accuracy: 0.9722222222222222
```

This output represents the accuracy of the trained SVM classifier on the testing data. The accuracy is approximately 0.972, which means the model correctly classifies the digits for about 97.2% of the samples in the testing set.

2.4 Naive Bayes Classifier

Naive Bayes Classifier is a probabilistic algorithm that is used for classification tasks. It is based on Bayes' theorem, which states that the probability of a hypothesis is updated when new evidence is observed. Naive Bayes Classifier assumes that the input features are independent of each other, which is why it is called "naive."

Here is the high-level algorithm for Naive Bayes Classifier:

1. Load the dataset and split it into training and testing sets.
2. Preprocess the data by scaling or normalizing the features.
3. Calculate the prior probabilities of each class.
4. Calculate the conditional probabilities of each feature given each class.

5. For each instance in the testing set, calculate the probability of each class using Bayes' theorem.
6. Predict the class with the highest probability.
7. Evaluate the performance of the model using metrics like accuracy, precision, recall, and F1 score.

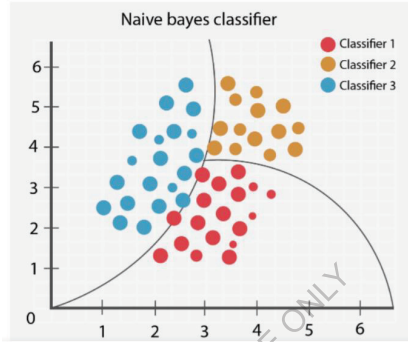


Figure 2.5: Naive Bayes Classifier

Here is an example of Naive Bayes Classifier used for binary classification:

In this example, the Naive Bayes Classifier creates decision boundaries (represented by the colored lines) that separate the two classes. The data points on either side of the decision boundaries are assigned a probability of belonging to their respective classes.

Here is an example of using Naive Bayes Classifier to classify the iris dataset:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load the iris dataset
iris = load_iris()

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2)
```

Create a Naive Bayes Classifier model and fit it to the training data

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

Make predictions for the testing data

```
y_pred = model.predict(X_test)
```

Evaluate the model's accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print('Accuracy:', accuracy)
```

In this example, we load the iris dataset and split it into training and testing sets. We then create a Naive Bayes Classifier model and fit it to the training data. Finally, we use the model to make predictions for the testing data and evaluate its accuracy.

Output:

```
Accuracy: 0.9666666666666667
```

The Naive Bayes Classifier achieved an accuracy of 96.67% on the iris dataset.

Naive Bayes Classifier is a simple yet powerful algorithm for classification tasks. It is easy to implement, computationally efficient, and can handle large datasets. While Naive Bayes Classifier assumes that the input features are independent of each other, it still performs well in practice and is widely used in various fields such as text classification, spam filtering, and sentiment analysis

2.5 Neural Networks

A Neural Network (NN) is a class of machine learning algorithms inspired by the structure and function of the human brain. It is composed of multiple layers of interconnected nodes (called neurons) that work together to learn patterns in the data and make predictions.

A Neural Network consists of an input layer, one or more hidden layers, and an output layer. Each layer contains a set of neurons that process the input data and produce an output. Here is an example of a simple Neural Network architecture:

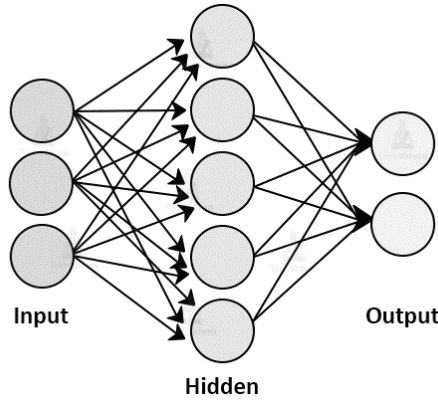


Figure 2.6: Neural Network architecture

In this example, the Neural Network has one input layer, one hidden layer, and one output layer. The input layer takes in the data, the hidden layer processes the data and learns the patterns, and the output layer produces the final prediction.

Activation Functions:

Each neuron in a Neural Network applies an activation function to the input data and produces an output. The most commonly used activation functions are:

Sigmoid function:

The sigmoid function is a non-linear activation function that maps any input value to a value between 0 and 1. It is defined as:

$$\text{sigmoid}(x) = 1 / (1 + e^{(-x)}) \quad (2.2)$$

where x is the input to the neuron.

Rectified Linear Unit (ReLU) function:

The ReLU function is a non-linear activation function that maps any input value to a value between 0 and infinity. It is defined as:

$$\text{ReLU}(x) = \max(0, x) \quad (2.3)$$

where x is the input to the neuron.

Softmax function:

The softmax function is a non-linear activation function that maps the output of a Neural Network to a probability distribution over the classes. It is defined as:

$$\text{softmax}(x) = e^{(x_i)} / \sum(e^{(x_j)}) \text{ for } j = 1 \text{ to } n \quad (2.3)$$

where x is the input to the output layer, and n is the number of classes.

Neuron Computation:

1. The computation of a neuron can be described as follows:
2. The neuron receives an input vector x of size n .
3. Each input x_i is multiplied by a weight w_i and summed together with a bias b .
4. The resulting value z is passed through an activation function f to produce the output y .

The mathematical formula for computing a neuron can be written as:

$$z = \sum(w_i * x_i) + b$$

$$y = f(z)$$

Backpropagation Algorithm:

The Backpropagation algorithm is used to train a Neural Network by adjusting the weights and biases of the neurons. It works by computing the gradient of the loss function with respect to the weights and biases, and updating them using the gradient descent algorithm.

The Backpropagation algorithm can be summarized as follows:

1. Forward propagate the input data through the network and calculate the output.
2. Calculate the loss between the predicted output and the actual output.
3. Backward propagate the loss through the network and calculate the gradient of the loss with respect to the weights and biases.
4. Update the weights and biases using the gradient descent algorithm.
5. Repeat steps 1-4 for a certain number of epochs or until convergence.

Neural Networks are a highly effective class of machine learning algorithms that can learn patterns in data and make accurate predictions. They consist of layers of interconnected nodes or neurons, which apply activation functions to input data and produce outputs. The weights and biases of the neurons are adjusted using the Backpropagation algorithm, which computes the gradient of the loss function and updates the parameters using gradient descent. By iteratively adjusting the parameters during training, Neural Networks can achieve high accuracy on a wide range of tasks.

FOR AUTHOR USE ONLY